

제03장.

- 오라클 접속에서 새로운 유저 생성

```
create user user03 identified by pass;  
grant connect, resource, dba to user03;
```

- USER03 접속 SQL 워크시트에서 출력 활성화

```
SET SERVEROUTPUT ON;
```

- OPEN ... FOR을 이용한 학생관리

오라클 PL/SQL에서 OPEN cursor_name FOR 구문은 일반적인 CURSOR 선언과 달리 OPEN ... FOR를 사용하면 실행 시점에 쿼리를 결정하거나 결과를 외부로 전달(REF CURSOR)할 수 있는 유연성을 갖게 된다.

- 특정 학과를 입력 받아 교수 테이블에서 해당 교수 목록과 인원수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX01(  
    i_dept IN professors.dept%TYPE,  
    o_cur OUT SYS_REFCURSOR,  
    o_count OUT NUMBER)  
AS  
BEGIN  
    OPEN o_cur FOR SELECT * FROM professors WHERE dept LIKE '%' || i_dept || '%';  
    SELECT COUNT(*) INTO o_count FROM professors WHERE dept LIKE '%' || i_dept || '%';  
END EX01;
```

```
--1. 바인드 변수 선언 (SQL Developer 명령어)  
var cur refcursor;  
var cnt number;  
--2. 프로시저 실행 (변수이름 앞에 콜론을 붙여야 함)  
exec ex01(' ', :cur , :cnt);  
--3. 바인드 변수 결과 출력  
print cur;  
print cnt;
```

- 특정 학과명을 입력 받아 학생 뷰에서 해당 학생 목록과 인원수를 출력한다.

```
create view view_students as  
select s.*, p.dept as pdept, pname from professors p, students s where pcode=advisor;
```

```
create or replace PROCEDURE EX02(  
    i_dept IN students.dept%TYPE,  
    o_cur OUT SYS_REFCURSOR,  
    o_count OUT NUMBER)  
AS  
    v_sql VARCHAR2(1000);  
    v_dept VARCHAR2(100) := '%' || i_dept || '%';  
BEGIN  
    v_sql := 'SELECT * FROM view_students WHERE dept LIKE :1';  
    OPEN o_cur FOR v_sql USING v_dept;  
    SELECT COUNT(*) INTO o_count FROM view_students WHERE dept LIKE v_dept;  
END EX02;
```

3) 검색키와 검색어를 입력 받아 학생 뷰에서 해당 학생 목록과 인원수를 출력한다.

```
create or replace PROCEDURE EX03(
    i_key IN VARCHAR2,
    i_word IN VARCHAR2,
    o_cur OUT SYS_REFCURSOR,
    o_count OUT INT)
AS
    v_word VARCHAR2(100);
    v_sql VARCHAR2(1000);
BEGIN
    v_word := '%' || i_word || '%';
    v_sql := 'SELECT * FROM view_students WHERE ' || i_key || ' LIKE :1';
    OPEN o_cur FOR v_sql USING v_word;
    DBMS_OUTPUT.PUT_LINE(v_sql);
    --EXECUTE IMMEDIATE 프로그램 실행도중 실시간으로 SQL 문장을 실행
    v_sql := 'SELECT count(*) FROM view_students WHERE ' || i_key || ' LIKE :1';
    EXECUTE IMMEDIATE v_sql INTO o_count USING v_word;
END EX03;
```

4) 정렬키, 정렬방식, 페이지 번호, 한 페이지 사이즈를 입력받아 해당 페이지 학생 목록과 전체 인원수를 출력한다.

```
(select s.*, row_number() over (order by scode) as rn from view_students s) -- 테이블은 반드시 별명을 지정한다.
```

```
select * from
(select s.*, row_number() over (order by scode) as rn from view_students s)
where rn between 1 and 5;
```

```
create or replace PROCEDURE EX04(
    i_order IN VARCHAR2,
    i_desc IN VARCHAR2,
    i_page IN INT,
    i_size IN INT,
    o_cur OUT SYS_REFCURSOR,
    o_count OUT INT)
AS
    v_sql VARCHAR2(1000);
    v_tbl VARCHAR2(1000);
    v_start INT := (i_page - 1) * i_size + 1;
    v_end INT := (i_page * i_size);
BEGIN
    v_tbl := '(SELECT s.*, ROW_NUMBER() OVER(ORDER BY ' || i_order || ' ' || i_desc || ') AS rn FROM view_students s)';
    v_sql := 'SELECT * FROM ' || v_tbl || ' WHERE rn BETWEEN :1 AND :2';
    DBMS_OUTPUT.PUT_LINE(v_sql);
    OPEN o_cur FOR v_sql USING v_start, v_end;
    v_sql := 'SELECT count(*) FROM view_students';
    EXECUTE IMMEDIATE v_sql INTO o_count;
END EX04;
```

```
var cur refcursor;
var cnt number;
exec ex04('sname','desc',1, 3, :cur, :cnt);
print cur;
print cnt;
```

- OPEN ... FOR을 이용한 강좌관리

1) 검색키와 검색어를 입력 받아 강좌 뷰에서 해당 학생 목록과 인원수를 출력한다.

```
create view view_courses as
  select c.*, p.dept as pdept, pname from professors p, courses c where pcode=INSTRUCTOR;
```

```
create or replace PROCEDURE EX05(
  i_key IN VARCHAR2,
  i_word IN VARCHAR2,
  o_cur OUT SYS_REFCURSOR,
  o_count OUT INT)
AS
  v_word VARCHAR2(100) := '%' || i_word || '%';
  v_sql VARCHAR2(1000);
BEGIN
  v_sql := 'SELECT * FROM view_courses WHERE ' || i_key || ' LIKE :1';
  DBMS_OUTPUT.PUT_LINE(v_sql);
  OPEN o_cur FOR v_sql USING v_word;
  v_sql := 'SELECT count(*) FROM view_courses WHERE ' || i_key || ' LIKE :1';
  EXECUTE IMMEDIATE v_sql INTO o_count USING v_word;
END EX05;
```

2) 정렬키, 정렬방식, 페이지 번호, 한 페이지 사이즈를 입력받아 해당 페이지 강좌 목록과 전체 강좌수를 출력한다.

```
(select c.*, row_number() over(order by lcode) as rn from view_courses c);
```

```
select * from
(select c.*, row_number() over(order by lcode) as rn from view_courses c)
where rn between 1 and 5;
```

```
create or replace PROCEDURE EX06(
  i_order IN VARCHAR2,
  i_desc IN VARCHAR2,
  i_page IN INT,
  i_size IN INT,
  o_cur OUT SYS_REFCURSOR,
  o_count OUT INT)
AS
  v_sql VARCHAR2(1000);
  v_tbl VARCHAR2(1000);
  v_start INT := (i_page-1) * i_size +1;
  v_end INT := (i_page * i_size);
BEGIN
  v_sql := 'SELECT * FROM ';
  v_tbl := '(SELECT c.*, ROW_NUMBER() OVER(ORDER BY ' || i_order || ' ' || i_desc || ') AS rn FROM view_courses c)';
  v_sql := v_sql || v_tbl || ' WHERE rn BETWEEN :1 AND :2';
  DBMS_OUTPUT.PUT_LINE(v_sql);
  OPEN o_cur FOR v_sql USING v_start, v_end;
  v_sql := 'SELECT COUNT(*) FROM view_courses';
  EXECUTE IMMEDIATE v_sql INTO o_count;
END EX06;
```

```
var cur refcursor;
var cnt number;
exec ex06('lname','desc',1, 3, :cur, :cnt);
print cur;
print cnt;
```

- OPEN ... FOR을 이용한 수강신청관리

1) 학생번호를 입력하면 학생정보와 해당 학생이 수강 신청한 강좌정보를 출력한다.

```
create view view_enrollments_cou as
select e.scode, e.edate, e.grade, c.* from courses c, enrollments e where c.lcode=e.lcode;
```

```
create or replace PROCEDURE EX07(
    i_scode IN enrollments.scode%TYPE,
    cur_students OUT SYS_REFCURSOR,
    cur_enrollments OUT SYS_REFCURSOR)
AS
    v_count INT;
    v_sql VARCHAR2(1000);
BEGIN
    SELECT COUNT(*) INTO v_count FROM view_students WHERE scode = i_scode;
    IF v_count > 0 THEN
        v_sql := 'SELECT * FROM view_students WHERE scode = :1';
        OPEN cur_students FOR v_sql USING i_scode;
        v_sql := 'SELECT * FROM view_enrollments_cou WHERE scode = :1';
        OPEN cur_enrollments FOR v_sql USING i_scode;
    END IF;
END EX07;
```

```
var cur1 refcursor;
var cur2 refcursor;
exec ex07('92414029', :cur1 :cur2);
print cur1;
print cur2;
```

2) 강좌번호를 입력하면 강좌정보와 이 강좌를 수강 신청한 학생 정보 출력한다.

```
create view view_enrollments_stu as
select e.lcode, s.* from enrollments e, view_students s where e.scode=s.scode;
```

```
create or replace PROCEDURE EX08(
    i_lcode IN enrollments.lcode%TYPE,
    cur_courses OUT SYS_REFCURSOR,
    cur_enrollments OUT SYS_REFCURSOR)
AS
    v_count INT;
    v_sql VARCHAR2(1000);
BEGIN
    SELECT COUNT(*) INTO v_count FROM view_courses WHERE lcode=UPPER(i_lcode);
    IF v_count > 0 THEN
        v_sql := 'SELECT * FROM view_courses WHERE lcode = :1';
        OPEN cur_courses FOR v_sql USING UPPER(i_lcode);
        v_sql := 'SELECT * FROM view_enrollments_stu WHERE lcode = :1';
        OPEN cur_enrollments FOR v_sql USING UPPER(i_lcode);
    END IF;
END EX08;
```

```
var cur1 refcursor;
var cur2 refcursor;
exec ex08('C301', :cur1, :cur2);
print cur1;
print cur2;
```

- 수강신청, 수강취소 관리

1) 학번, 강좌번호를 입력 받아 학번과 강좌번호를 유, 무를 체크, 수강신청 중복 체크 후 수강 신청 추가 작업을 한다.

```
insert into students(scode, sname, dept, year, birthday, advisor)
values('95454001', '홍길동', '건축', 3,'73/01/01','509');
```

```
insert into students(scode, sname, dept, year, birthday, advisor)
values('95454002', '심청이', '건축', 3,'73/01/03','509');
```

```
insert into students(scode, sname, dept, year, birthday, advisor)
values('95454003', '강감찬', '건축', 3,'73/01/04','509');
```

```
insert into students(scode, sname, dept, year, birthday, advisor)
values('95454004', '성춘향', '건축', 3,'73/01/05','509');
```

```
insert into courses(lcode, lname, hours, room, instructor, capacity, persons)
values('A101','건축학개론', 3 ,'506', '509', 100, 80);
```

```
insert into courses(lcode, lname, hours, room, instructor, capacity, persons)
values('A102','건축설계', 3 ,'506', '509', 100, 80);
```

```
insert into courses(lcode, lname, hours, room, instructor, capacity, persons)
values('A102','건축설계', 3 ,'506', '509', 100, 80);
```

```
create or replace PROCEDURE EX09(
    i_scode enrollments.scode%TYPE,
    i_lcode enrollments.lcode%TYPE)
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('학번:' || i_scode || ', 강좌번호:' || i_lcode);
    INSERT INTO enrollments(scode, lcode, edate) VALUES(i_scode, i_lcode, SYSDATE);
    DBMS_OUTPUT.PUT_LINE('수강신청 성공');
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        CASE
            WHEN SQLCODE = -2291 THEN
                DBMS_OUTPUT.PUT_LINE('학번이나 강좌번호가 없습니다.');
```

```
-- 학번이 없는 경우
exec ex09('95454000', 'A101');
-- 강좌가 없는 경우
exec ex09('95454001', 'A100');
-- 수강신청 성공한 경우
exec ex09('92414001', 'A101');
-- 이미 수강 신청한 경우
exec ex09('92414001', 'A101');
```

2) 학번, 강좌번호를 입력 받아 학번과 강좌번호를 유, 무를 체크, 수강신청 유, 무 체크 후 수강 취소 작업을 한다.

```
CREATE OR REPLACE PROCEDURE EX10(
    i_scode IN enrollments.scode%TYPE,
    i_lcode IN enrollments.lcode%TYPE)
AS
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM students WHERE scode=i_scode;
    IF v_count =0 THEN
        DBMS_OUTPUT.PUT_LINE('학번이 없습니다.');
```

```
    ELSE
        SELECT COUNT(*) INTO v_count FROM courses WHERE lcode=i_lcode;
        IF v_count = 0 THEN
            DBMS_OUTPUT.PUT_LINE('강좌번호가 없습니다.');
```

```
        ELSE
            SELECT COUNT(*) INTO v_count FROM enrollments WHERE scode=i_scode AND lcode=i_lcode;
            IF v_count = 0 THEN
                DBMS_OUTPUT.PUT_LINE('수강 신청한 강좌가 없습니다.');
```

```
            ELSE
                DELETE FROM enrollments WHERE scode=i_scode AND lcode=i_lcode;
                DBMS_OUTPUT.PUT_LINE('수강 취소 성공.');
```

```
            END IF;
        END IF;
    END IF;
END EX10;
```

```
-- 학번이 없는 경우
exec ex10('95454000', 'A101');
```

```
-- 강좌가 없는 경우
exec ex10('95454001', 'A100');
```

```
-- 수강 취소가 성공한 경우
exec ex10('95454001', 'A101');
```

```
-- 취소할 강좌가 없는 경우
exec ex10('95454001', 'A101');
```

3) 강좌 테이블의 수강학생 인원수를 실제 인원수로 수정한다.

```
create or replace PROCEDURE EX11
AS
    CURSOR cur_enrollments IS SELECT lcode, COUNT(*) AS cnt FROM enrollments GROUP BY lcode;
BEGIN
    UPDATE courses SET persons = 0;
    FOR rec IN cur_enrollments LOOP
        DBMS_OUTPUT.PUT_LINE('강좌번호:' || rec.lcode || ',수강신청인원수:' || rec.cnt);
        UPDATE courses SET persons = rec.cnt WHERE lcode = rec.lcode;
    END LOOP;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('에러 발생: ' || SQLERRM);
END EX11;
```

```
exec ex11();
```

4) 수강신청 프로시저(ex09)에서 수강신청이 성공한 경우 강좌 테이블의 수강인원수를 1 증가 시킨다.

```
create or replace PROCEDURE EX09 ( ... ) AS
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM students WHERE scode=i_scode;
    ..
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('강좌번호가 없습니다.');
```

```
    ELSE
        SELECT COUNT(*) INTO v_count FROM enrollments WHERE scode=i_scode AND lcode=i_lcode;
        IF v_count = 1 THEN
            DBMS_OUTPUT.PUT_LINE('이미 수강 신청된 강좌입니다.');
```

```
        ELSE
            INSERT INTO enrollments(scode, lcode, edate) VALUES(i_scode, i_lcode, '98/03/06');
            UPDATE courses SET persons=persons+1 WHERE lcode=i_lcode;
            COMMIT;
            DBMS_OUTPUT.PUT_LINE('수강신청 완료.');
```

```
        END IF;
    END IF;
END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END EX09;
```

```
exec ex09('95454001', 'A101');
```

5) 수강취소 프로시저(ex10)에서 수강취소가 성공한 경우 강좌 테이블의 수강인원수를 1 감소시킨다.

```
create or replace PROCEDURE EX10 ( ... ) AS
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM students WHERE scode=i_scode;
    ...
    SELECT COUNT(*) INTO v_count FROM courses WHERE lcode=i_lcode;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('강좌번호가 없습니다.');
```

```
    ELSE
        SELECT COUNT(*) INTO v_count FROM enrollments WHERE scode=i_scode AND lcode=i_lcode;
        IF v_count = 0 THEN
            DBMS_OUTPUT.PUT_LINE('수강 신청한 강좌가 없습니다.');
```

```
        ELSE
            DELETE FROM enrollments WHERE scode=i_scode AND lcode=i_lcode;
            UPDATE courses SET persons=persons-1 WHERE lcode=i_lcode;
            DBMS_OUTPUT.PUT_LINE('수강취소 성공.');
```

```
            COMMIT;
        END IF;
    END IF;
END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
END EX10;
```

```
exec ex10('95454001', 'A101');
```